EDM2

Virtual Control Program Interface specification v1

EDM/2 preamble

This is the *Virtual Control Program Interface specification v1* as last published by Phar Lap and Quarterdeck in 1989. It is republished here with inherent permission from the original publishers but you should keep in mind that it is an intellectual property of Phar Lap, Quarterdeck and other committee members and cannot be changed or reused without their permission and is not governed by the Attribution-Share Alike 3.0 licence like the rest of the EDM/2 Wiki.

The content of the documentation is unchanged apart from the following:

- Sales, technical help, download and contact information has been removed as none of those services are available anymore.
- Miniscule changes have been made to the text, spelling errors, formatting errors and possible copy and paste errors have been fixed and/or noted.

Outside of formatting changes, adding Wikilinks and graphic improvements, the document shall be left as it is. It should be noted that VCPI is an outdated system that was never actually supported by OS/2 but has some technical relevance to the DOS subsystem, see VCPI.

Virtual Control Program Interface

Version 1.0 June 12, 1989

Copyright Phar Lap Software, Inc., 1987-1989 Copyright Quarterdeck Office Systems, 1987-1989

This specification may be duplicated provided it is reproduced in its entirety, including the above copyright notice.

VCPI SPONSORS

- A.I. Architects, Inc., Peabody, MA
- Lotus Development Corp., Cambridge, MA
- Phar Lap Software, Inc., Cambridge, MA
- Quadram, Norcross, GA
- Qualitas, Inc., Bethesda, MD
- Quarterdeck Office Systems, Santa Monica, CA
- Rational Systems, Inc., Natick, MA

VCPI Co-ordinators

- Phar Lap Software, Inc., Robert Moote

- Quarterdeck Office Systems, Daniel Spear

Version 1.0 Specification Revision History

June 12, 1989
Final revision approved by attendees of the VCPI Developer's Conference, held at the offices of Phar Lap Software on April 25, 1989. Attendees included the following VCPI sponsors: A.I. Architects, Inc., Lotus Development Corporation, Phar Lap Software, Inc., Qualitas, Inc., Quarterdeck Office Systems, Rational Systems, Inc.; and the following observers: Compaq Computer Corporation, Intel Corporation.

May 8, 1989
Revision incorporating modifications agreed upon at the VCPI Developer's Conference. This revision was distributed for approval to all conference attendees.

March 31, 1989
A more detailed and explicit revision of the December 1987 specification, drafted for review at the VCPI Developer's Conference.

December 21, 1987
First revision of the specification, written by Phar Lap Software, Inc., and Quarterdeck Office Systems.

1.0 Background

The Intel 80386 microprocessor has three fundamental operating modes. Real mode is provided for backward compatibility with existing 8086 programs. Protected mode allows programs written specifically for the 80386 to take advantage of the larger address space available. Virtual 8086 (V86) mode, like real mode, is used to run 8086 programs. However, V86 mode runs under the control of a protected mode operating environment. This provides certain advantages, chiefly the ability to enable the paging hardware of the 80386 and thus run multiple 8086 programs simultaneously, and also the ability to make arbitrary physical memory available within the V86 address space of an 8086 program. (The 80486 microprocessor provides the same architecture and operating modes as the 80386, thus software written for the 80386 runs without modification on an 80486).

The capabilities of the 8086 have spawned the creation of several new kinds of control programs that can run under MS-DOS on a 386 machine. To date, these programs fall into three basic categories:

1. protected mode run-time environments, which allow an application program to execute in protected mode under MS-DOS;
2. EMS emulators, which use V86 mode to make all the memory on the machine available to 8086 programs which use EMS (Lotus/Intel/Microsoft Expanded Memory Specification) memory, and,
3. multitasking environments which use V86 mode to multitask 8086 MS-DOS programs, while still giving each 8086 program a full 640Kb of physical memory (or more, if they use EMS memory). Such multitasking environments typically run in conjunction with a separate EMS emulator, or implement the EMS interface as part of the multitasker. For the remainder of this specification, the terms "DOS-Extender," "EMS emulator," and "multitasker," respectively, will be used to refer to these program categories.

Since these control programs run under MS-DOS, it is desirable to make them compatible with each other, so that users don't have to turn off one control program in order to run an application under another control program. The purpose of this document is to specify an interface that allows

these classes of control programs to coexist successfully. This interface is called the Virtual Control Program Interface, or VCPI.

2.0 Areas of Conflict

There are a variety of resource contention issues which must be solved to allow multiple 386 control programs to coexist. The section below summarizes each of these issues.

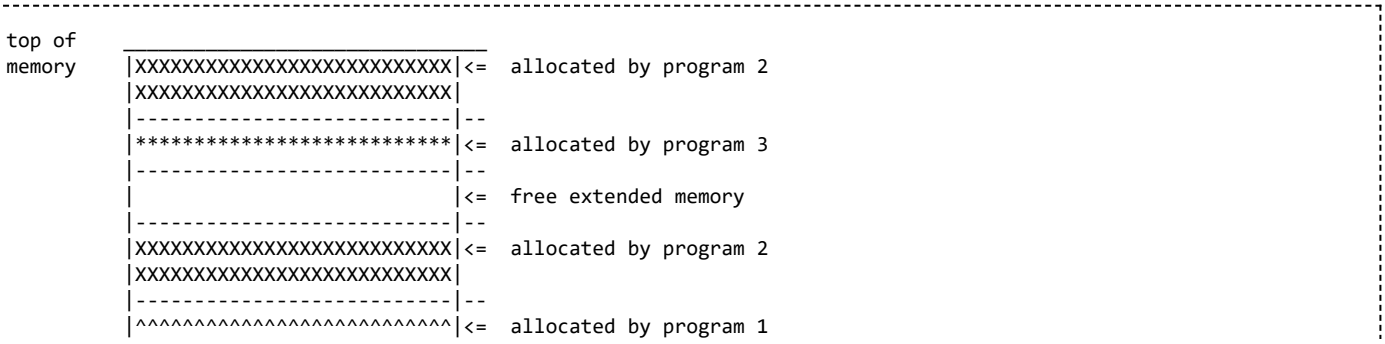
2.1 Extended Memory Allocation

Because DOS only knows about memory below one MB, there is no universally used method in the DOS environment for allocating and freeing extended memory (memory above one MB). The following existing techniques are available for allocating extended memory on both 80286 and 80386 PCs:

1. Memory can be allocated "top down" by taking over the BIOS extended memory size system call (INT 15h function 88h) and reporting less extended memory available than is actually present on the machine. This method allocates a contiguous block of memory starting at the top of extended memory and growing downward. See Appendix A.1 for a description of top down allocation.
2. Memory can be allocated "bottom-up" by taking over the PC reboot interrupt (INT 19h) and installing a signature block and allocation size marker in the interrupt handler, and boot block with an allocation size marker at one megabyte. This is commonly referred to as the VDISK technique, since it is the method employed by the original IBM VDISK driver. This method allocates a contiguous block of memory starting at one megabyte and growing upward. See Appendix A.2 for a description of bottom-up allocation.
3. If an XMS (Microsoft Extended Memory Specification) drive is installed, extended memory can be allocated by making calls to the XMS driver. The XMS driver reserves extended memory for an XMS memory pool using top-down allocation, and the allocates memory out of the XMS memory pool. The XMS specification may be read here as eXtended Memory Specification v2.0

A program that wishes to allocate extended memory must first determine what extended memory is free (not in use by another program). It does this by calling INT 15h function 88h to obtain the top of free extended memory, and by checking the signature of INT 19h and the boot block at one megabyte for any bottom-up memory allocation, thus obtaining the bottom of free extended memory.

The program can then allocate its own chunk of extended memory, either top down by taking over INT 15h, or bottom-up by taking over INT 19h. The top-down allocation method is preferable because it is simpler and more robust. The diagram below shows an example in which three programs which use extended memory are installed. Program 1 (the first program installed) allocates some extended memory bottom-up, program 2 allocates two chunks of memory, one bottom-up and one top-down, and program 3 allocates memory top-down.



```
1 MB   => |--------------------------|--
          |//////////////////////////|<=  BIOS ROM, screen, etc.
640 KB => |--------------------------|--
          |\\\\\\\\\\\\\\\\\\\\\\\\\\|<=  DOS memory
          |__________________________|__
```

Note that only the last installed program which allocates extended memory top-down or bottom-up (the first program in the interrupt chain for INT 15h or INT 19h) can dynamically change its memory allocation. In the example above, only program 2 can dynamically modify its bottom-up memory allocation, and only program 3 can dynamically modify its top-down memory allocation.

It is possible for VCPI memory, XMS memory and extended memory all to be available on a single PC. (This can occur, for example, if a VCPI driver is installed and is given a portion of extended memory, an XMS driver is installed and even another chunk of extended memory, and the rest of extended memory is left free.) Therefore, programs which can use extended memory may wish to look for all three sources of memory. The recommended order of use of the different memory allocation methods is: (1) VCPI memory, (2) XMS memory, and, (3) extended memory.

2.2 Mode Switching

All 386 control programs need to do some processing in protected mode, but also need to allow standard 8086 code (such as other applications, DOS, and the BIOS) to execute. Therefore, they must switch between 80386 protected mode and either real mode or V86 mode.

If two control programs both switch between protected mode and real mode, no special problems are posed. If, however, a control program switches between V86 mode and protected mode, it is impossible for a second control program to switch into protected mode. This is because programs running in V86 mode are executing at the least privileged level of the 80386 processor, so system-level operations will cause processor exceptions, which result in the first control program regaining control. Thus the first control program must provide a method to allow subsequent control programs to switch to protected mode and back to V86 mode again.

2.3 8259A Interrupt Controller

The standard interrupt vectors used on the 386 PC for hardware interrupts IRQ0 - IRQ7 are vectors 08h - 0Fh. Unfortunately, those vectors are also used by the 80386 processor for certain processor exceptions. For this reason, some 386 control programs reprogram the 8259A interrupt controller chip to relocate IRQ0 - IRQ7 to different interrupt vectors. Information about reprogramming of the 8259A must be made available to all cooperating 386 control programs.

2.4 Privileged Instructions

When the 386 processor is executing in V86 mode, it is executing at privilege level 3 (the least privileged level), and privileged instructions cause an exception. These include instructions which load or store 386 system registers, including the control registers, the debug registers, the test registers, GDTR, LDTR, and TR. Access to some of these registers is needed in V86 mode; others only need to be modified as part of a mode switch to protected mode.

In addition, a number of instructions cause a general protection exception if the I/O privilege level (IOPL) is not 3 when executing in V86 mode. These instructions must be made available, either by setting IOPL to 3, or by emulating the instructions.

3.0 Interface Summary

Many of the problems listed above occur only when one control program uses V86 mode. EMS emulators run in V86 mode and are widely used on 386 PCs, and an EMS interface is typically used or provided by multitaskers. In addition, one of the primary functions performed by EMS emulators is memory allocation.

For these reasons, the VCPI interface is defined as a superset of the EMS version 4.0 interface (an EMS 4.0 driver must be provided underneath the VCPI interface; a copy of the EMS 4.0 interface specification may be read here as *Expanded Memory Specification v4.0*). If an EMS emulator with the VCPI interface is installed, a multitasker and/or an arbitrary number of DOS-Extender programs which use the VCPI interface can be run. Control programs which utilize the VCPI interface provided by the EMS emulator will be referred to as "clients" for the remainder of this document. The control program providing the VCPI interface (and the EMS 4.0 interface) will be referred to as the "server" for the remainder of this document.

A server is installed at boot time, and takes over a block of extended memory which becomes the expanded memory pool from which it allocates EMS pages. When the server is installed, it can be given all the extended memory on the computer, or extended memory can be "partitioned" into expanded memory and extended memory (i.e., some extended memory is taken by the server for the expanded memory pool, and the rest is left free, available for use by other programs).

Clients can obtain memory by making requests of the server. Memory allocated through the VCPI interface comes out of the memory pool owned by the server (i.e., expanded memory), and must be in the physical address space of the computer (e.g., it must not be memory on a separate expanded memory board). If any extended memory is left free by the server, clients can allocate that extended memory in the usual way.

The VCPI interface is implemented with calls to functions provided by the server. Calls need to be made both in V86 mode and in protected mode. The V86 mode calls are implemented through the standard INT 67h EMS interface. The protected mode calls are made with FAR procedure calls to an entry point in the server. The protected mode entry point is obtained via an initialization on call to the server.

Both the server and each client program maintain their own set of system tables (GDT, LDT, IDT, page tables, etc.) Each client program must initialize a memory space in its page tables which is identical to a portion of the memory space used by the server, in order to provide a common ground for communication with the server. When the client calls the server to switch to protected mode, the client's system tables are set up as part of the mode switch. Likewise, when the client calls the server to switch back to V86 mode, the server's system tables are set up.

The server must leave A20 (address line 20 on a PC) enabled at all times, in both V86 mode and in protected mode.

The client must always use the services of the server to switch modes. It should never, for example, implement its own switch back to V86 mode, because the server's system tables must be set up when executing in V86 mode.

4.0 V86 Mode Program Interface

The V86 mode interface is provided through the EMS interrupt (INT 67h), with an EMS function code in AH of DEh, and a VCPI function code in AL. (Note that any EMS driver which does not provide VCPI should return either code 84h (bad function code) for this EMS call, since it is not a defined function number for the LIM 4.0 EMS specification.) If the function succeeds, AH is returned as zero; otherwise, there is an error code in AH. Except as noted for each call, all other registers are unchanged on output.

If an undefined VCPI subfunction code is passed in AL, the server returns an either code in AH; the recommended value is 8Fh (bad subfunction code).

The IOPL-sensitive privileged instructions must be available in V86 mode. This means the server must either set the IOPL to 3, or emulate these instructions. Other privileged instructions need not be emulated as part of the VCPI interface. Instead, access to certain system registers is provided via system calls (see Section 4.4).

4.1 Presence Detection

The presence of a server which provides the VCPI interface is detected by first using one of the standard detection checks to see if an EMS driver is installed, and then making the EMS call below to check for the presence of VCPI.

```
INT 67h
AX = DE00h     VCPI Presence Detection
==============================================================================
INPUTS:        None
------------------------------------------------------------------------------
OUTPUTS:       If VCPI is present:
                    AH = 0
                    BL = 0 (VCPI minor version number, in binary)
                    BH = 1 (VCPI major version number, in binary)
               If not present:
                    AH = nonzero (recommended value:  84h)
```

If the server is installed but no EMS memory has been allocated, the server may be executing in real mode rather than V86 mode. The detection call should return success in real mode as well as V86 mode (so that it is possible to detect a VCPI server that is currently turned off, and also for backward compatibility with clients that make this call in real mode); however, for the rest of the VCPI interface to be enabled, the processor must be executing in V86 mode. Therefore, the following VCPI detection sequence is recommended:

1. Check to see if the processor is an 80386 or 80486.
2. Check to see if an EMS driver is installed.
3. Allocate one EMS page. This will turn on the EMS driver if it is off, switching the processor to V86 mode.
4. Make the above VCPI detection call.

The EMS page allocated in step 3 must not be deallocated again until the client is done using the VCPI interface; otherwise, the server could turn off again, setting the processor back to real mode and disabling the VCPI interface.

Appendix A.3 contains a code sequence demonstrating VCPI detection.

4.2 Interface Initialization

The interface initialization process involves creating a common ground between the server and the client that allows them to communicate in protected mode, and providing any other information that the client may need to complete its own initialization. Since the server and the client each have separate page tables, GDT, LDT, and IDT, in order to communicate in protected mode the client must set up a region in its page tables that maps memory used by the server, and must initialize segment descriptors in its GDT to point to code and data regions in the server.

The calls below are used to initialize the interface between the server and the client.

```
INT 67h
AX = DE01h     Get Protected Mode Interface
===========================================================================
INPUTS:        ES:DI = pointer to 4K page table buffer
               DS:SI = pointer to three descriptor table entries in the
                       client's GDT, the first to be set up as the
                       protected mode code segment in the server, and the
                       additional two entries to be used as desired by
                       the server when it is called in protected mode
---------------------------------------------------------------------------
OUTPUTS:       AH = 0
               DI = advanced to point to the first unused page table
                    entry in the buffer (could be advanced by 4K, if
                    server initializes entire page table)
               EBX = offset in code segment of protected mode entry point
```

The Get Protected Mode Interface call is used to set up a common frame of reference which allows the client to make calls to the server in protected mode. The call initializes a page table which the client must set up as its 0th page table. This page table must map at least one megabyte of memory starting at linear address zero: the real mode address space provided in V86 mode. In addition, the server can map any additional memory above one megabyte (up to a total of four megabytes, since there is just one page table) which it requires to provide the protected mode interface. While the mappings for the first megabyte must be identical for both the client and the server (so the client will be able to set up segments to access its V86 memory from protected mode), memory above one megabyte can be mapped as desired by the server (e.g., if the server's code and data is mapped at a high address in its own page tables, it would simply map the same code and data just above one megabyte in the client's 0th page table).

For backward compatibility, both the server and the client must clear the software-defined bits (bits 9 - 11) in the page table entries it copies into the client's page table; all other bits should be preserved the client must not copy its 0th page table to a different physical page after this call is made. This is so the server can always access the client's 0th page table. The client must never modify any bits other than the software-defined bits (9-11) in the page table entries which are initialized by this call.

In addition to initializing the 0th page table, this call also fills in a GDT entry in the client's GDT. This segment is used by the client when calling the server in protected mode. The segment must be set up as a code segment, and must reside in the linear address space set up in the 0th page table

by this call. The entry point of the interface routine (its offset within the code segment) is returned in EBX. Two additional entries in the client's GDT, immediately following the GDT entry for the code segment, are available to the server for its own use. Typically at least one of these GDT segments is used by the server as a data segment.

When the client calls the server in protected mode, it makes a FAR call to a USE32 segment, using the entry point provided by this call. Since the server has initialized the code segment descriptor, it can, of course, set it up any way it desires, but it is responsible for executing a USE32 FAR return when it terminates, regardless of the actual use type of the code segment. Thus, when the server is called in protected mode, it will have CS pointing to the code segment which is set up by this call. It can obtain selectors for the two additional GDT entries which it set up by adding 0008h and 0010h, respectively, to the segment selector value in CS. The server must preserve all segment registers across a protected mode VCPI call.

```
INT 67h
AX = DE02h     Get Maximum Physical Memory Address
================================================================================
INPUTS:        None
--------------------------------------------------------------------------------
OUTPUTS:       AH = 0
               EDX = PHYSICAL address of the highest 4K memory page that
                     could ever be allocated.
```

The Get Maximum Physical Memory Address call is used by the client to initialize its memory management data structures. To do this, it may need to know the physical address of the highest 4K memory page that could ever be allocated by the Allocate 4K Page call specified below. For backward compatibility, both the server and the client must mask the 12 LSBs of the physical page address (returned in EDX) to zero.

4.3 Memory Allocation Calls

The memory allocation calls are used to allocate and free 4K pages of memory. The server allocates these pages out of its EMS memory pool. The client can also do its own allocation of extended memory or XMS memory, if any is available.

```
INT 67h
AX = DE03h     Get Number of Free 4K Pages
================================================================================
INPUTS:        None
--------------------------------------------------------------------------------
OUTPUTS:       AH = 0
               EDX = number of free 4K pages
```

This call returns the total number of 4K pages available to be allocated out of the server's expanded memory pool.

Note that this is the total amount of free memory available to all tasks on the system. The client should allocate this memory only on an as needed basis. In addition, in a multitasking environment, this number does not reflect any memory restrictions placed on the client task by the multitasker. The client should, during initialization, use the EMS call Get Number of Pages (INT 67h function 3 (AH=42h)) to obtain the number of unallocated EMS pages available, and then limit its usage of memory allocated through the server to that amount. (Note that the size of an EMS page is 16K, and the size of a page allocated through the VCPI interface is 4K.)

```
INT 67h
AX = DE04h     Allocate a 4K Page
===============================================================================
INPUTS:        None
-------------------------------------------------------------------------------
OUTPUTS:       If success:
                    AH = 0
                    EDX = PHYSICAL address of allocated 4K page
               If failure:
                    AH = nonzero (recommended value:  88h)
                    EDX = contents modified
```

This call allocates a 4K page of memory. For backward compatibility, both the server and client must make the 12 LSBs of the physical page address (returned in EDX) to zero. The client is responsible for freeing all memory allocated with this call before terminating execution.

```
INT 67h
AX = DE05h     Free a 4K Page
===============================================================================
INPUTS:        EDX = PHYSICAL address of 4K page to free
-------------------------------------------------------------------------------
OUTPUTS:       If success:
                    AH = 0
               If failure:
                    AH = nonzero (recommended value:  8Ah)
```

This call frees a page of memory previously allocated with subfunction 04h. For backward compatibility, both the server and the client must mask the 12 LSBs of the physical page address (passed in EDX) to zero.

```
INT 67h
AX = DE06h     Get Physical Address of 4K Page in First Megabyte
===============================================================================
INPUTS:        CX = page number (linear address of page SHR 12)
-------------------------------------------------------------------------------
OUTPUTS:       If success:
                    AH = 0
                    EDX = PHYSICAL address of 4K page
               If invalid page number:
                    AH = nonzero (recommended value:  8Bh)
```

This call returns the physical address of a 4K page in the first megabyte of the V86 mode linear address space. For backward compatibility, both the server and the client must mask the 12 LSBs of the physical page address (returned in EDX) to zero.

This call is useful for obtaining the address of memory which is allocated and mapped into the first megabyte with the standard EMS calls. While this information can also be extracted from the page table obtained with the Get Protected Mode Interface system call, that page table is a "snapshot" of the server's page tables at the time the system call was made, and would not have the correct addresses for any EMS memory mapped in after the Get Protected Mode Interface call was made.

4.4 System Register Access Calls

When executing in V86 mode, any attempt to load or store system registers causes a general protection exception. The system calls below provide access to certain system registers in V86 mode.

```
INT 67h
AX = DE07h     Read CR0
===============================================================================
INPUTS:        None
```

```
-------------------------------------------------------------------------------
OUTPUTS:        AH = 0
                EBX = CR0 value
```

This call returns the current value of the CR0 register.

```
INT 67h
AX = DE08h      Read Debug Registers
===============================================================================
INPUTS:         ES:DI = pointer to array of 8 DWORDS, DR0 first in
                        array, DR4 and DR5 not used
-------------------------------------------------------------------------------
OUTPUTS:        AH = 0
```

This call stores the values of the debug registers into an array in memory.

```
INT 67h
AX = DE09h      Load Debug Registers
===============================================================================
INPUTS:         ES:DI = pointer to array of 8 DWORDS, DR0 first in array,
                        DR4 and DR5 not used
-------------------------------------------------------------------------------
OUTPUTS:        AH = 0
```

This function loads the debug registers with the values in the array.

4.5 8259A Interrupt Vector Mapping Calls

The default PC interrupt vector assignments for hardware interrupts are vectors 08h - 0Fh for interrupts IRQ0 - IRQ7, and vectors 70h - 77h for interrupts IRQ8 - IRQ15. The 80386 processor uses interrupt vectors 08h - 0Fh for processor exceptions, which are important to 386-specific software. Therefore, some clients find it desirable to reprogram the 8259A interrupt controller to map the hardware interrupt vectors elsewhere in the interrupt space. The server, as the first control program in place, is the repository of information on what the current vector mappings are. Client programs should query the server during initialization to determine the current mappings. If they have been changed from the PC default mappings, the client is obligated not to change them again. If they are still set to the PC defaults, the client can reprogram the interrupt controller and then inform the server what the new mappings are.

```
INT 67h
AX = DE0Ah      Get 8259A Interrupt Vector Mappings
===============================================================================
INPUTS:         None
-------------------------------------------------------------------------------
OUTPUTS:        AH = 0
                BX = 1st vector mapping for master 8259A (IRQ0-IRQ7)
                CX = 1st vector mapping for slave 8259A (IRQ8-IRQ15)
```

This function returns the interrupt vectors generated by the 8259A interrupt controller chips when a hardware interrupt occurs. If there is no slave 8259A to generate IRQ8-IRQ15 (because the software is running on a 386 accelerator card plugged into a PC or a PC/XT), then the value returned in CX is unspecified.

Note that, for programs who wish to take over hardware interrupts in V86 mode, other considerations may apply when executing in a multitasking environment.

```
INT 67h
AX = DE0Bh      Set 8259A Interrupt Vector Mappings
===============================================================================
```

```
INPUTS:        BX = vector mapping for master 8259A (IRQ0-IRQ7)
               CX = vector mapping for slave 8259A (IRQ8-IRQ15)
               Interrupts disabled
-------------------------------------------------------------------------------
OUTPUTS:       AH = 0
```

This call is used by the client to inform the server if it remaps the 8259A interrupt controllers. The client must leave interrupts disabled from the time it begins to reprogram the interrupt controllers until after it makes this function call to inform the server of the remapping.

Note that the server must leave interrupts disabled until after it has logged the vector mappings provided by the call. In particular, it must not "reflect" the interrupt back to V86 mode before recording the information, in case there is another program in the INT 67h chain which enables interrupts.

If the client remaps the 8259A interrupt controller, it is responsible for restoring the original interrupt controller mappings when it terminates.

4.6 Switch to Protected Mode

```
INT 67h
AX = DE0Ch     Switch From V86 Mode to Protected Mode
===============================================================================
INPUTS:        ESI = linear address of data structure in first megabyte,
                     containing values to load into system registers
               Interrupts disabled
-------------------------------------------------------------------------------
OUTPUTS:       GDTR, IDTR, LDTR, TR loaded
               Interrupts disabled
               Control transferred to specified FAR entry point
               SS:ESP must have at least 16 BYTEs of space on it, and the
                      protected mode entry point is required to set up its
                      own stack before re-enabling interrupts
               EAX = contents modified
               ESI = contents modified
               DS, ES, FS, GS = contents modified
               All other registers unmodified
```

This call switches to protected mode, sets up all the system tables for the client, and transfers control to the specified entry point in the client. Interrupts must remain disabled for the entire duration of the switch. The ESI register contains a linear address of a data structure in the first megabyte; because the page table mappings for the first megabyte are identical in both the server's and the client's page tables, this address can be used by the server both before and after reloading the CR3 register.

The data structure pointed to by ESI looks like:

```
     10 (DWORD) CS:EIP too transfer control to
     0E (WORD)  selector value to load into TR
     0C (WORD)  selector value to load into LDTR
     08 (DWORD) linear address in first megabyte of 6-byte value to load into IDTR
     04 (DWORD) linear address in first megabyte of 6-byte value to load into GDTR
ESI->00 (DWORD) new value to load into CR3
```

Note that the server must load GDTR before loading the LDTR and TR (IDTR can be loaded at any time, since interrupts are disabled). The GDTR and IDTR must be loaded with the 32-bit forms of the LGDT and LIDT instructions, not the 24-bit forms provided for backward compatibility with the 80286 processor. The server must clear the task busy bit in the TSS descriptor table entry in the client's GDT before loading TR (which must be loaded with an LTR instruction, not a task

switch). Note that, since the client's GDT could reside anywhere in the 4 GB linear address space, the TSS descriptor table entry cannot be modified (and the LDTR and TR registers cannot be loaded) until after CR3 is loaded with the client's value, and the data segment used to modify the TSS descriptor must have a base address of zero and a limit of 4 GB. The server must not reload any segment registers after it loads the GDTR with the client's GDT address; note that the ability to modify GDTR and LDTR without reloading segment registers relies on the segment descriptor caching performed by the processor.

The server's code that implements the switch must reside in the linear region that was initialized with the Get Protected Mode Interface call (function DE01h), since that is the only linear region that is mapped identically in both the server's and the client's page tables.

When the client gets control, the stack is still set to whatever stack the server was using (because SS has not been reloaded, the descriptor cache register for SS still points to the server's stack segment). The client must reload all segment registers and set up its own stack before enabling interrupts. This is a requirement since an interrupt handler will attempt to save and restore segment registers, and the values left in the segment registers after the switch are not necessarily valid segments for the client.

5.0 Protected Mode Interface

The protected mode interface is a FAR entry point in the server, which is obtained during initialization with the Get Protected Mode Interface system call (INT 67h function DE01h). The client makes a FAR call to a USE32 segment when it calls the server entry point, and the server must execute a USE32 FAR return when it terminates.

An EMS function code of DEh is passed in AH, and a VCPI function code is passed in AL. If the function succeeds, AH is returned as zero; otherwise, there is an error code in AH. Except as noted for each call, all other registers are unchanged on output.

If an undefined VCPI subfunction code is passed in AL, the server returns an error code in AH; the recommended value is 8Fh (bad subfunction code).

5.1 Memory Allocation Calls

The memory allocation calls are used to allocate and free 4K pages of memory the server allocates these pages out of its expanded memory pool. The client can also do its own allocation of extended memory or XMS memory, if any is available.

```
INT 67h
AX = DE03h     Get Number of Free 4K Pages
================================================================================
INPUTS:        None
--------------------------------------------------------------------------------
OUTPUTS:       AH = 0
               EDX = number of free 4K pages
```

This call returns the total number of 4K pages available to be allocated out of the server's expanded memory pool.

Note that this is the total amount of free memory available to all tasks on the system. The client should allocate this memory only on an as needed basis. In addition, in a multitasking environment, this number does not reflect any memory restrictions placed on the client task by the

multitasker. The client should, during initialization, use the EMS call Get Number of Pages (INT 67h function 3 (AH = 42h)) to obtain the number of unallocated EMS pages available, and then limit its usage of memory allocated through the server to that amount. (Note that the size of an EMS page is 16K, and the size of a page allocated through the VCPI interface is 4K.)

```
INT 67h
AX = DE04h     Allocate a 4K page
================================================================================
INPUTS:        None
--------------------------------------------------------------------------------
OUTPUTS:       If success:
                    AH = 0
                    EDX = PHYSICAL address of allocated 4K page
               If failure:
                    AH = nonzero (recommended value:  88h)
                    EDX = contents modified
```

This call allocates a 4K page of memory. For backward compatibility, both the server and client must mask the 12 LSBs of the physical page address (returned in EDX) to zero. The client is responsible for freeing all memory allocated with this call before terminating execution.

```
INT 67h
AX = DE05h     Free a 4K Page
================================================================================
INPUTS:        EDX = PHYSICAL address of allocated 4K page
--------------------------------------------------------------------------------
OUTPUTS:       If success:
                    AH = 0
               If failure:
                    AH = nonzero (recommended value:  8Ah)
```

This call frees a page of memory previously allocated with subfunction 04h. For backward compatibility, both the server and the client must mask the 12 LSBs of the physical page address (passed in EDX) to zero.

5.2 Switch to V86 Mode

```
INT 67h
AX = DE0Ch     Switch From Protected Mode to V86 Mode
================================================================================
INPUTS:        Top of stack looks as described below
               Interrupts disabled
               SS:ESP must be in the first megabyte of linear memory
               DS = segment selector that maps the entire linear address space
                    that was obtained with subfunction 01h, Get Protected Mode
                    Interface (i.e., its base address is 0, and its limit is
                    the size of the address space obtained with the Get
                    Protected Mode Interface call)
--------------------------------------------------------------------------------
OUTPUTS:       GDTR, IDTR, LDTR, TR loaded with server's values
               Interrupts disabled
               Control transferred to specified FAR entry point
               SS:ESP and all segment registers loaded with values on stack
               EAX = destroyed
               All other registers unmodified
```

This call switches from protected mode back to V86 mode, after setting up all of the server's system tables. The top of stack looks as follows on input to this function:

```
         28 (DWORD) GS value
         24 (DWORD) FS value
         20 (DWORD) DS value
         1C (DWORD) ES value
         18 (DWORD) SS value
```

```
         14 (DWORD) ESP value
         10 (DWORD) reserved for EFLAGS value
         0C (DWORD) CS value
         08 (DWORD) EIP value
SS:ESP-> 00 (DWORD) return address from FAR call to USE32 segment
```

The first 8 bytes at SS:ESP are the return address pushed by the FAR CAlL instruction; since this call never returns, the return address is irrelevant and should be discarded. It is the responsibility of the server to fill in an appropriate EFLAGS value (e.g., VM bit set, IOPL = 3, IF bit cleared) before executing the IRETD to return to V86 mode. Interrupts must remain disabled for the entire duration of the switch, and the interrupt enable bit must be off in the EFLAGS value which is stored on the stack before executing the IRETD which switches back to V86 mode. The server's code that implements the switch must reside in the linear region that was initialized with the Get Protected Mode Interface call (function DE01h), since that is the only linear region that is mapped identically in both the server's and the client's page tables.

If the client does any task switching, it should clear the TS bit in CR0 before making this call. Otherwise, the TS bit will still be set after the switch to V86 mode, and a floating point coprocessor instruction in V86 mode will cause an exception 7 trap to the server.

Implementation Notes

A.1 Top-Down Extended Memory Allocation

The BIOS extended memory size system call is used to determine the upper limit of extended memory which is not in use by other programs. The following code fragment gets the physical address of one byte past the end of available extended memory in the EAX register:

```
      xor  eax,eax             ; clear MSBs in EAX
      mov  ah, 88h             ; get size of extended memory in KB
      int  15h
      shl  eax, 10             ; convert it address of top of memory
      add  eax, 100000h
```

To allocate extended memory top down, simply install an INT 15h handler which passes all functions except 88h through to the previous INT 15h handler. If an INT 15h function 88h call is made, return a value which reduces the size of extended memory by the amount of memory you wish to allocate.

A.2 Bottom-Up Extended Memory Allocation

Bottom-up extended memory allocation is not recommended, as it is more complicated and less versatile than top-down allocation. However, programs that wish to allocate extended memory must first check to see if any other programs have allocated extended memory using the bottom-up technique. Bottom-up extended memory allocation is done by marking the amount of memory allocated in two locations. If there is any extended memory on the system (if the BIOS call documented in Appendix A.1 does not return zero), it is necessary to check BOTH of these locations to see if any bottom-up extended memory has been allocated. If the two locations show different amounts of allocated memory, the safest thing to do is terminate with an error message. Failing that, the larger of the two values found should be used. The values may differ in the two

locations because there are several programs available which use bottom-up allocation but which do not correctly update both locations, and also because the DOS PRINT command in DOS 3.3 or later wipes out the allocation information in one of the two locations.

The first location is the segment pointed at by the INT 19h vector (the reboot interrupt). If an appropriate signature is present at a specific offset in this segment, then there is a 24-bit address of the first free byte of extended memory at another specific offset in the segment. The following code fragment obtains the physical address of the first free byte of extended memory and loads it into EAX:

```
data segment
     vdisk_sig db 'VDISK V'       ; signature string
     SIG _ LEN equ $ - vdisk_sig  ; signature string length
     SIG_VEC equ 19h              ; interrupt vector for signature
     SIG_OFFS equ 12h             ; offset of signature string in segment
     LO_OFFS equ 2Ch              ; offset of low word of 24-bit value
     HI_OFFS equ 2Eh              ; offset of high byte of 24-bit value
     data ends

assume    cs:code,ds:data
code      segment use16

     xor ax,ax                 ; get segment addr in ES
     mov es,ax
     mov ax,es:(SIG_VEC * 4) + 2
     mov es,ax

     mov di,SIG_OFFS           ; branch of signature not present
     lea si,vdisk_sig
     mov ex,SIG_LEN
     cld
     repe cmpsb
     jne short no_alloc

     xor eax,eax               ; get addr of bottom of extended memory
     mov al,es:HI_OFFS         ; into EAX
     shl eax,16
     mov ax,es:LO_OFFS
     jmp short done            ; all done

no_alloc:
          mov eax,100000h      ; no bottom-up memory allocated

done:                          ; bottom of extended memory is now in EAX
code ends
```

The second location to check is a boot block located at one megabyte. The following C code fragment retrieves the allocation value from the boot block and saves it in the variable "bot_ext" (note that this code should NOT be executed if there is no extended memory on the system, since it attempts to read memory at one megabyte):

```
long bot_ext;        /* addr of 1st free byte of extended memory */
char buf [0x20];     /* buffers boot block */
#define SIGOFF 0x03  /* offset of signature in boot block */
#define SIZEOFF 0x1E /* offset of size word in boot block */

/*
* Read boot block at one megabyte by calling an assembly language
* routine (not shown) which either uses the BIOS block move call to
* read from extended memory, or does it in some other way.
*/

read_ext(0x100000, buf, sizeof(buf));

/*
* Now check for the correct signature in the boot block, and if
* present, extract the bottom of available extended memory.
*/
```

```
if (memcmp(&buf[SIGOFF], "VDISK", 5) != 0)
     bot_ext = 0x100000;
else
{
     bot_ext = 01;
     memcpy(&bot_ext, &buf[SIZEOFF], 2]; /* get KB addr */
     bot_ext = bot_ext << 10;            /* convert to byte addr */
```

A.3 Sample VCPI Detection Code

The following code sequence should be used to detect the presence of the VCPI interface:

```
data segment
emm_name db 'EMMXXXX0',0      ; device name for EMM
data ends

     assume      cs:code,ds:data
code segment use16
;
; First check to make sure we are on an 80386 or 80486
;
     pushf               ; save flags
     xor ax,ax           ; try to put a zero word into flags and then
     push ax             ; get result back into AX
     popf
     pushf
     pop ax
     and ax,0F000h       ; if high 4 bits of flags are all 1's, it's an
     cmp ax,0F000h       ; 8086/8088/80186/80188
     jne short not_386
     mov ax,0F000h       ; Now try to set the high 4 bits of the flags
     push ax             ; and then get the result back in AX
     popf
     pushf
     pop ax
     and ax,0F000h        ; if none of the high 4 bits are set, it's an
     jz short not_386     ; 80286 -- otherwise, it's a 386 or 486
     popf                 ; restore flags
     jmp short ems_check  ; go to next check
not_386:
     popf                 ; restore flags
     jmp no_vcpi          ; not a 386 or 486 -- VCPI not present

ems_check:
;
; Check for EMS driver present
;
     lea dx,emm_name      ; open EMM device, read-only
     mov ax,3D00h
     int 21h
     jc short no_vcpi     ; branch if error
     mov bx,ax            ; get device information
     mov ax,4400h
     int 21h
     jc short no_vcpi     ; branch if error
     test dx,80h          ; branch if file (not device)
     jz short no_vcpi
     mov ax,4407h         ; check output status
     int 21h
     push ax              ; save return code from IOCTL call
     mov ah,3Eh           ; close file
     int 21h
     pop ax               ; restore IOCTL return code
     cmp al,0FFh          ; branch if status not device ready
     jne short no_vcpi

; Now allocate an EMS page to make sure the EMS emulator is turned on.
; Note that we must save the handle for this allocated page and free it
; again eventually, although this is not shown in the code fragment.
;
    mov bx,1           ; allocate 1 EMS page
    mov ah,43h
    int 67h
    cmp ah,0          ; branch if error
    jne short no_vcpi
; save handle returned in DX to be freed later
```

```
; Now make VCPI detection call
;
    mov ax,0DE00h        ; is VCPI there?
    int 67h
    cmp ah,0             ; branch if not there
    jne short no_vcpi
; OK, the VCPI interface is present and the server is turned on.
no_vcpi:                 ; VCPI interface not present
code ends
```

GLOSSARY

4K page
: A four-kilobyte page, the page size allocated by calls to the VCPI interface. A 4K page is always aligned on a four-kilobyte physical address boundary.

A20
: Address line 20. On AT and MCA architectures, there is hardware to truncate physical addresses to 20 bits (one megabyte). This is known as "disabling A20," and is for compatibility with 8086 programs which rely on address wraparound at one megabyte. To use extended memory, it is necessary to "enable A20."

bottom-up allocation
: Allocation of extended memory starting at one megabyte and growing upward. This allocation method is used by older VDISK device drivers.

client
: A program which uses the VCPI interface

expanded memory
: Memory allocated by an EMS emulator

extended memory
: Physical memory above one megabyte.

EMS page
: A 16-kilobyte page, the page size allocated by calls to an EMS driver. An EMS page is always allocated on a four-kilobyte physical address boundary.

real mode address space
: Linear addresses below one megabyte (note that this does not necessarily correspond to physical addresses below one megabyte)

server
: A program which provides the VCPI interface

top-down allocation
: Allocation of extended memory starting at the top of memory and growing downward. This is the recommended technique for allocating extended memory.

Retrieved from "https://www.edm2.com/index.php?title=Virtual_Control_Program_Interface_specification_v1&oldid=50223"